

Forecasting Cryptocurrency Prices Using Hybrid Long Short – Term Memory and Long Short – Term Memory Variants

Thuan Dinh Nguyen

thuannnd@uit.edu.vn

Vietnam National University, Ho Chi Minh City

Research Article

Keywords: Cryptocurrency, Deep Learning, Long Short-Term Memory

Posted Date: November 27th, 2024

DOI: <https://doi.org/10.21203/rs.3.rs-5374266/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Forecasting Cryptocurrency Prices Using Hybrid Long Short – Term Memory and Long Short – Term Memory Variants

Thuan Nguyen Dinh^{1,2}, Nhi Le Thi Ai^{1,2*}†, Nhut Nguyen Minh^{2,3†}

^{1,2,3}University of Information Technology, Ho Chi Minh City, 700000,
VietNam.

*Corresponding author(s). E-mail(s): 19521963@gm.uit.edu.vn ;

Contributing authors: thuand@uit.edu.vn; nhutnm.17@grad.uit.edu.vn;

†These authors contributed equally to this work.

Abstract

Nowadays, virtual currency has become mainstream and attracts many investors and the environment. People spend large amounts of money in the hope that their trades are buying which occurs when the price of a cryptocurrency falls and selling when it rises in search of a profit. However, due to the volatility of these currencies, it is difficult for investors to decide whether to buy, sell or hold. So in this research, we build a valuable daily data set of virtual currencies, especially Bitcoin, Ethereum, and Binance. We rely on it to build deep learning models along with model improvement techniques and model fusion techniques to improve model performance. We use basic regulatory response measurements to evaluate the model's results, i.e. MAE, MAPE, MSE, and RMSE.

Keywords: Cryptocurrency, Deep Learning, Long Short-Term Memory

1 Introduction

In recent years, virtual currency has become one of the most prominent subjects among investors and brokers, attracting significant attention. The interest in investing in such a crucial virtual currency is attributable to its considerable volatility over time, stemming from price fluctuations. Consequently, achieving high accuracy in forecasting

virtual currency price movements necessitates the construction of deep learning models and integrating algorithms to yield precise prediction results.

This study selects the most popular cryptocurrencies, including Bitcoin, Ethereum, and Binance, to conduct a comprehensive future value prediction test based on their market value. It is crucial to note that each coin possesses distinct features and applications, warranting careful consideration by potential users before engaging in cryptocurrency investments.

The research entails an investigation utilizing deep learning models, emphasizing exploring variations to devise improved methods for processing lengthy and complex time series data. Specifically, we constructed predictive models based on Long Short-Term Memory (LSTM), currently acknowledged as one of the most powerful and widely adopted deep learning models. Moreover, we explored various LSTM variants, such as Vanilla LSTM, Stacked LSTM, Bi-LSTM, CNN-LSTM, ConvLSTM, Multiplicative LSTM, Peephole LSTM, Encoder-Decoder LSTM, Deep LSTM Network, and TimeDistributed LSTM. Additionally, we integrated a Hybrid LSTM-CNN model and applied a combination of Bi-LSTM and CNN-LSTM, subsequently refining the Hybrid LSTM-CNN model to enhance prediction outcomes. Finally, we analyzed and evaluated the model results on the baseline regression index, i.e., MAE, MAPE, MSE, and RMSE.

2 Related work

Patel et al. proposed a hybrid cryptocurrency prediction method, focusing on Litecoin and Monero cryptocurrencies. The proposed model is based on neural network architecture regression using LSTM and GRU layers. The data in their study contains daily Litecoin data from August 24, 2016, to February 23, 2020, and Monero data from January 30, 2015, to February 23, 2020, with the average price: average, opening price, closing price, high and low price, and trading volume. The reported experiments have demonstrated that the proposed association model outperforms LSTM networks giving promising results [19].

In 2023, Authors Seabe P.L, Moutsinga C.R.B, and Pindza E conducted a study on forecasting cryptocurrencies using deep learning. In it, the researchers performed three types of Recurrent Neural Networks (RNNs), namely Long Short-Term Memory(LSTM), Gated Recurrent Unit (GRU), and Bi-Directional LSTM (Bi-LSTM) for forecasting cryptocurrencies, including Bitcoin(BTC), Ethereum (ETH), and Litecoin (LTC). Then use the Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE) measures to check the inter-algorithm accuracy. The results show that the Bi-LSTM algorithm predicts better than LSTM and GRU. In addition, the researchers suggest that future development directions should focus on factors that affect the price of cryptocurrencies, such as the media and the number of transactions [22].

In 2021, Liquan Shan and his colleagues researched an algorithm model combining Bi-Directional LSTM and convolutional neural network (CNN) to predict water shortage value based on factors affecting. In addition, the author also incorporates an attention mechanism based on the final advantage of the cell state of BiLSTM or used

to align the input window state in the current step using the hidden state of Bi-LSTM. The RMSE, MAE, NAPE, and PCC measures to evaluate the performance of the predictive models gave the result that the combined model results gave an accurate and higher result when forecasting [24].

In 2021, Zhifang Liao, Peng Lan, and their colleagues developed methods to predict the development trend of the spread of COVID-19. The study authors have experimented with the basic model of SIRVD. However, the results cannot be processed in the long-term data set, so they have proposed a deep learning predictive model of recurrent neural network and its variants: Vanilla LSTM, Stacked LSTM, Bi-LSTM, and GRU. Finally, using two measures, RMSE and NRMSE, to evaluate the model's performance, the results show that it can improve the prediction by 50% compared to the pure method and adjust the short-term forecast, and the medium-term gives better results [17].

In 2021, a study conducted by Changchun Cai, Yuan Tao, Tianqi Zhu, and Zhixiang Deng introduced a short-term load forecasting framework based on a multi-layer stacked two-way Long Short-Term Memory (LSTM) neural network. The proposed approach combines backward computation, forward computation, and a feedback computation mechanism to address the interdependencies between past and future power load time series information. Deep learning training captures the correlations among historical loads, while the network's multi-layered stacking structure effectively manages power load data. Real-world data was employed to evaluate the proposed method, and the results demonstrate its ability to extract dynamic features from the data and make accurate predictions [8].

In 2020, Lei Fu introduced a model known as Deep LSTM with peephole connections (DPLSTM) to improve load prediction accuracy in distributed systems. Traditional prediction models are known for their time-consuming nature, complexity, and limited accuracy. The load sequence exhibits stochastic, non-static, non-linear, and continuously oscillating characteristics. The proposed DPLSTM model was evaluated using real-world data to address these challenges. The experimental results indicate that DPLSTM performs remarkably in predicting load information, yielding predictions that closely align with the actual load values. This model outperforms existing load forecasting methods, showcasing its efficacy [12].

In 2019, Sima Siami-Namini and her colleagues conducted a comparative analysis of two types of Recurrent Neural Networks (RNNs) -Long Short-Term Memory (LSTM) and Bi-directional LSTM (BiLSTM) - for sequence forecasting in the context of time. The study's primary objective was to explore the potential improvement in predictability by incorporating additional training layers into the LSTM architecture. BiLSTM models were examined, which process input data from left to proper to left. The findings revealed that the BiLSTM-based model outperformed the traditional LSTM-based model and the ARIMA model regarding prediction accuracy. However, it was observed that BiLSTM models took longer to reach equilibrium compared to LSTM-based models [25].

In 2019, Anita Yadav, CK Jha, and Aditi Sharan conducted a study on optimizing the Long Short-Term Memory (LSTM) model for time series prediction in the Indian stock market. LSTMs have effectively addressed complex time series prediction

challenges, such as capturing long-term trends, seasonal and cyclical fluctuations, and random noise. However, the performance of LSTM models is significantly influenced by various hyperparameters, which require careful selection to achieve desirable outcomes. To bridge this research gap, the authors compiled a dataset from the Indian stock market and developed an LSTM model tailored to this context. They explored optimization techniques by comparing stateful and stateless models and adjusting the number of hidden layers. The findings demonstrated that a model configuration with a single hidden layer yielded superior results for most prediction scenarios. This configuration offered higher accuracy, more accessible training, and reduced risk of overfitting, making it the preferred choice [28].

3 Research Method

This study aims to use deep learning techniques, including LSTM and LSTM variants, to predict the price of three cryptocurrencies: BTC, ETH, and BNB. A procedure has been developed for model evaluation and research following the specific process illustrated in the figure 1 below:

3.1 The given prediction problem

We are interested in the multivariable time series prediction problem in this paper. To give a fully observed set of values $Y = \{y_1, y_2, \dots, y_T\}$ where $y_t \in \mathbb{R}^n$, and n is the size of the values, we aim to predict a range of future values over different periods. The output is the predicted result value of $y_{(T+h)}$ where h is the desired time to predict the upcoming outcome when the dataset $\{y_1, y_2, \dots, y_T\}$ is available. Thus, the same will also be available for the tuple $\{y_1, y_2, \dots, y_T, y_{(T+1)}\}$, so we constructed the input as the tuple.

The following matrix is defined as:

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \dots & x_1^d & x_1^1 & x_1^2 & \dots & x_1^n & x_2^1 & x_2^2 & \dots & x_2^n & \dots & x_T^1 & x_T^2 & \dots & x_T^n \end{bmatrix}$$

where $x_d = (x_d^1, x_d^2, \dots, x_d^n)$ is the attribute value of multiple values over T daily time, and $x^n = (x_1^n, x_2^n, \dots, x_T^n)$ are the properties of the time-independent variables such as High, Open, Low, Close, H-L, SMA₇, SMA₁₄, SMA₂₁ to be interpreted. This is specified in the dataset section to predict the outcome with the output being the predicted value of $y_{(T+h+1)}$.

3.2 Long Short – Term Memory

For the problem with the input sequence/time series, we usually know the RNN model. However, in the RNN model, the vanishing gradient occurs. The RNN cannot learn from the distant information for series data, especially time series, due to the vanishing gradient. In theory, RNNs can carry information from the previous to the last layers. However, in reality, the information can only be carried through a certain number of states, after which it will have a vanishing gradient; in other words, the model only

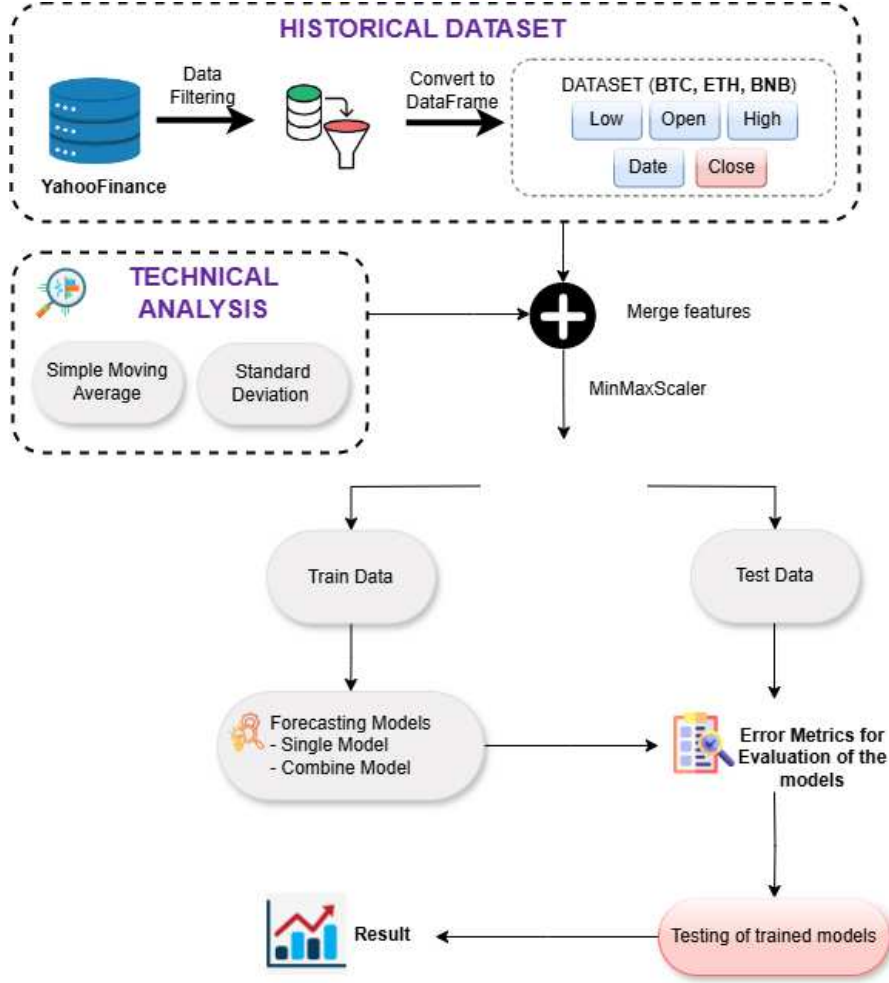


Fig. 1 The process of making prediction results

learns from the states near it; this is called short-term memory. Therefore, the Long Short-Term Memory (LSTM) model was born, described as follows: In the t state of the LSTM model:

- **Output:** c_t, h_t where c_t is the cell state and h_t is the hidden state.
- **Input:** $c_{(t-1)}, h_{(t-1)}, x_t$ where x_t is the input in the t state of the model and $c_{(t-1)}, h_{(t-1)}$ are the outputs of the previous layer.

Where σ is sigmoid function; multiplication is element-wise multiplication; addition is matrix addition, and f_t, i_t, o_t correspond to the forget gate, input gate, and output gate, respectively.

$$\text{Forget gate: } f_t = \sigma(U_f x_t + W_f h_{(t-1)} + b_f)$$

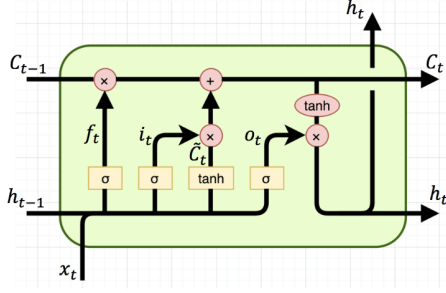


Fig. 2 Architectural model of LSTM

$$\text{Input gate: } i_t = \sigma(U_i x_t + W_i h_{(t-1)} + b_i)$$

$$\text{Output gate: } o_t = \sigma(U_o x_t + W_o h_{(t-1)} + b_o)$$

where $0 < f_t, i_t, o_t < 1$ and b_f, b_i, b_o are the bias terms.

The candidate cell state is computed as:

$$\tilde{c}_t = \tanh(U_v x_t + W_c h_{(t-1)} + b_c)$$

The cell state c_t is updated as:

$$c_t = f_t c_{(t-1)} + i_t \tilde{c}_t$$

The forget gate determines how much information to retain from the previous cell state, while the input gate decides how much to add from the candidate cell state and the hidden layer inputs of the previous layer.

Finally, the hidden state h_t is given by:

$$h_t = o_t \tanh(c_t)$$

The output gate decides how much to take from the cell state to output the hidden state. In addition, h_t is also used to calculate the output y_t for state t .

3.3 Stacked Long Short – Term Memory

The Stacked LSTM is a neural network architecture consisting of multiple LSTM layers stacked on each other. Each layer contains numerous LSTM cells. This deep structure of the Stacked LSTM model is a reliable approach for tackling complex sequence prediction tasks [18]. The formula for the Stacked LSTM algorithm with an input string of length n , represented by the vectors $(x_1, x_2, \dots, x_t)$, can be expressed as follows:

$$h_t = \text{LSTM}_k(h_{(t-1)}, \text{LSTM}_{(k-1)}(h_{(t-1)}, \dots, \text{LSTM}_1(h_{(t-1)}, x_t)))$$

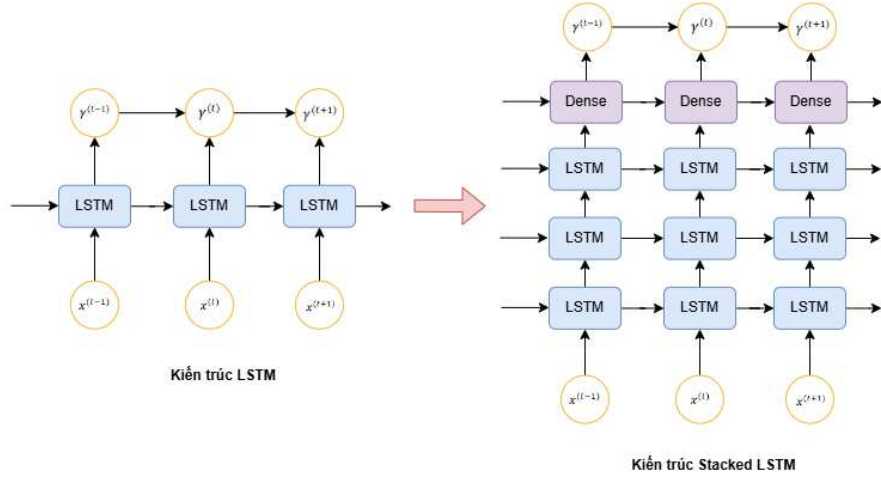


Fig. 3 The transformation between an LSTM model to a Stacked LSTM

where LSTM_1 denotes the first LSTM unit, and LSTM_k represents the last LSTM unit in the Stacked LSTM model. Furthermore, the combination of multiple LSTM layers helps mitigate the "vanishing gradient" issue, which occurs when the model's derivative approaches zero during training. This integration allows the model to learn intricate features and enhance prediction accuracy. Additionally, incorporating multiple LSTM layers facilitates faster convergence during training. However, due to the increased number of layers and neurons resulting from stacking multiple LSTM layers, the model's complexity is elevated. Regularization techniques such as dropout or weight decay can be utilized to address this challenge. These techniques involve randomly dropping elements and emphasizing necessary weights to alleviate overfitting in the model, as discussed in the study by Md Fazle Rabby in 2021 [20].

3.4 Multiplicative Long Short – Term Memory

3.4.1 Recurrent Neural Network (RNN)

Neural network systems cannot retain information from previous periods. In the context of time series modeling, memory retention is crucial in capturing the values at previous time steps to discern underlying patterns, as described by the time series law [4]. A multiplicative RNN (mRNN) is a specialized architecture that enables adaptable transitions based on the input received from an RNN.

3.4.2 Multiplicate LSTM (mLSTM)

The Multiplicative LSTM (mLSTM) is an architectural fusion of two models, namely LSTM and Multiplicative RNN (mRNN). In mLSTM, connections are established from the intermediate state of mRNN to all gate units within the LSTM. This integration combines the intricate input-dependent propagation of mRNN with the LSTM's ability to control and manage long-term information. The gated units of the LSTM facilitate transitions contingent upon more flexible input compared to traditional mRNN [9].

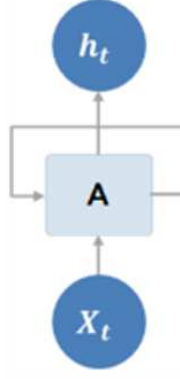


Fig. 4 The architecture of Recurrent Neural Network has a loop

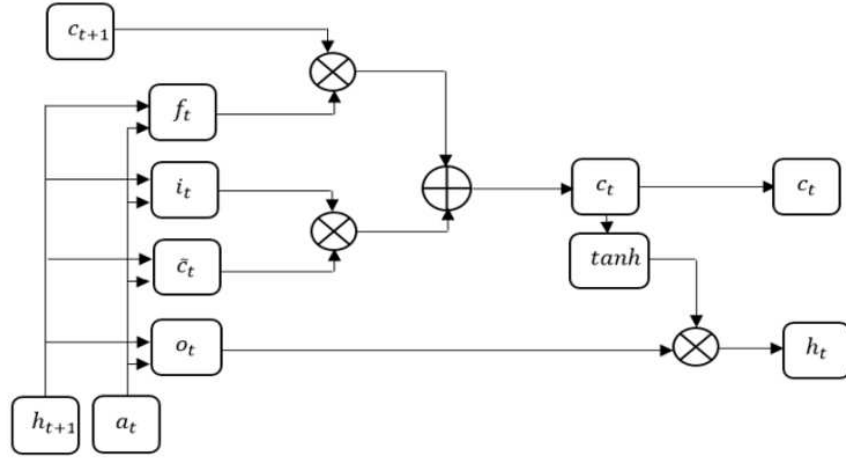


Fig. 5 Illustrated diagram of Multiplicate LSTM

3.5 Convolutional Neural Network Long Short – Term Memory

3.5.1 Convolutional Neural Network (CNN)

Convolutional neural networks (CNNs) are specific artificial neural networks for data processing. They can automatically learn and extract features from input data, eliminating the need for manually providing precomputed features [26].

In prediction problems, 1D convolution is utilized to process time series data. Consider a layer with input x_i^k and output y_j^k . The convolutional layer performs a matrix multiplication between the input from the preceding layer and a filter bank applied on x_i^k , while incorporating a normalization method [27]. The computation of the convolutional layer can be expressed using the following equation:

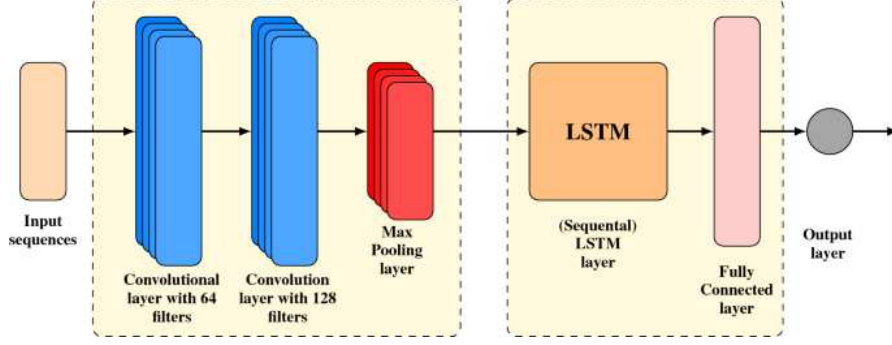


Fig. 6 Structure of the CNN-LSTM model

$$y_j^k = \sum_i (x_i^k \cdot w_{ij}^k) + b_j^k$$

3.5.2 Convolutional Neural Network LSTM (CNN-LSTM)

The Convolutional Neural Network LSTM (CNN-LSTM) is an architectural fusion that combines the strengths of Convolutional Neural Networks (CNN) and Long Short-Term Memory Networks (LSTM). In order to address the issue of overfitting that arises when the number of training iterations increases, Dropout layers are introduced to deactivate specific neurons within the CNN-LSTM model randomly. Conversely, short training sessions may result in suboptimal matching performance. Therefore, consideration must be given to the number of iterations and the appropriate training batch size to achieve optimal performance [1]. To tackle this problem, a residual solution is proposed as follows:

$$F(X) = H(X) - X$$

Here, H represents the mapping implemented by multiple stacked layers, and X denotes the input of the first layer. These layers are designed to approximate the residual function $F(X) = H(X) - X$ instead of directly mapping to $H(X)$. Furthermore, the combination of $F(X)$ and X is the network's mapping. This enables a simple model to approximate the output of numerous nonlinear layers towards zero, facilitating identity mapping and effectively resolving the degradation problem.

3.6 Hybrid Long Short – Term Memory – Convolutional Neural Network model

The hybrid LSTM-CNN model is a model that uses a hybrid architecture between LSTM and CNN. The LSTM-CNN model proposed in this paper uses the output vector of the multilayer LSTM model as the input vector for the CNN model and builds a CNN model on top of the multilayer LSTM model to extract the characteristics of the input text strings and increase the classification accuracy [16].

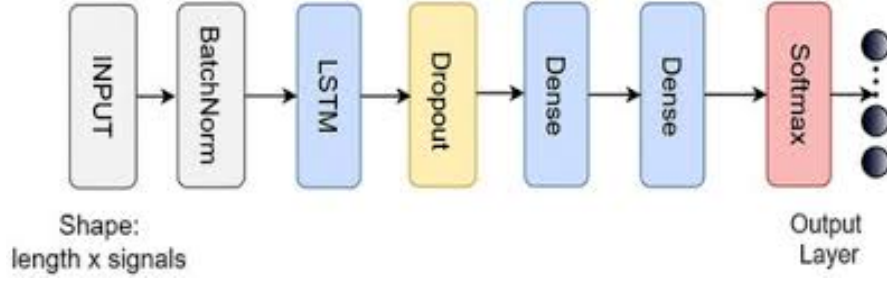


Fig. 7 Structure of the vanilla model LSTM

3.7 Vanilla Long Short – Term Memory

Vanilla Long Short-Term Memory (LSTM) is a specific type of recurrent neural network architecture designed to address the issue of vanishing gradients encountered in traditional recurrent neural networks (RNNs). Vanilla LSTM employs the LSTM network architecture and features a single hidden LSTM layer and an output layer for prediction [14].

In contrast to traditional LSTM, which utilizes separate input and forget gates to store information in the cell state and hidden state during computation, vanilla LSTM relies solely on the previous state of the status window for it's forget, input, and output gates. As a result, vanilla LSTM can learn less complex relationships compared to the standard LSTM model. However, vanilla LSTM is commonly used for more straightforward problems due to its simpler structure and faster training process.

3.8 TimeDistributed Long Short-Term Memory

3.9 TimeDistributed layer

TimeDistributed is a wrapper class that applies sequential, recurrent, or convolutional layers over a long time series. With a unidirectional input and output along $T = 6$ days, the input $(x_0, \dots, x_5) \in \mathbb{R}^6$ and the output is: $(y_0, \dots, y_5) \in \mathbb{R}^6$, where $W_y, b_y \in \mathbb{R}$ are the weights. With the formula [21]:

$$y_t = \sigma(W_y x_t + b_y)$$

for each $t \in \{0, \dots, 5\}$. Note that $W_y \in \mathbb{R}$ and $b_y \in \mathbb{R}$ are identical across all t values.

3.10 TimeDistributed LSTM

The TimeDistributed LSTM combines the LSTM architecture with the TimeDistributed layer. By utilizing the TimeDistributed LSTM class, each time point in a time series is treated independently by an LSTM model. The outputs from each LSTM model are then aggregated to generate a time series prediction. This approach proves beneficial for addressing time series forecasting problems. Each element within the

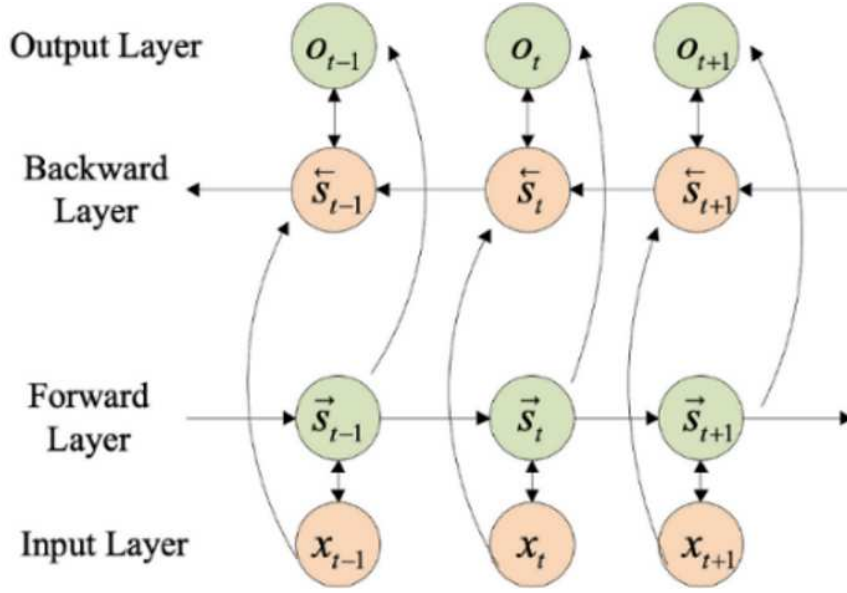


Fig. 8 BiLSTM model

sequence, processed by a separate LSTM model, can capture and retain patterns from past time steps while updating information regarding future price forecasts [6].

The backpropagation process in Timedistributed LSTM occurs on a per-time-step basis. This involves updating the weights between layers in the neural network to optimize the model, thereby improving the model's ability to approximate economic values. The backpropagation is performed iteratively, moving from the last time step to the first, allowing for the computation of gradients for the weighted loss function. These gradients are then used to adjust the weights between the model's layers, to enhance the model's predictive capabilities on new datasets.

3.11 Bidirectional Long Short – Term Memory

Within the LSTM model, we observe the sequential flow of data through the input, output, and forget gates that the input variable is transmitted horizontally from the input to the output. As a result, prediction errors accumulate gradually, leading to a sudden increase in proportion to the previous time steps within the prediction model. A proposed solution to address the error accumulation issue is the Bidirectional LSTM (Bi-LSTM) [15].

Bi-LSTM is a technique that enhances the LSTM by allowing the neural network to store sequence information in both forward and backward directions. The input data is processed in both directions, distinguishing Bi-LSTM from the underlying LSTM model. This bidirectional processing capability helps mitigate the error accumulation problem and enhances the model's overall performance.

BiLSTM consists of two layers of LSTM architecture; one is used to calculate hidden vectors from front to back, and the other is used to calculate hidden vectors

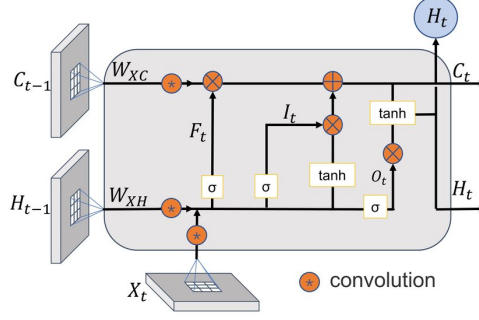


Fig. 9 ConvLSTM model

from back to front. These two layers define the output of the Bi-LSTM neural network. The LSTM bidirectional neural network can be written as:

$$s_t = f(Ux_t + Ws_{t-1})$$

$$s'_t = f(U'x_t + W's'_{t-1})$$

$$o_t = g(Vs_t + V's_t)$$

Where s_t , o_t , s'_t are the state variables of the hidden layer, the output layer, and the inverted layer at time t , respectively; x_t is the input vector; g and f are activation functions; V , W , and U are weight matrices from the hidden layer to the output layer, hidden layer to hidden layer, and input layer to hidden layer, respectively; V' , W' , and U' are inverse weight matrices, respectively.

3.12 Convolutional Long Short – Term Memory

The ConvLSTM model is a variant of the conventional LSTM model that combines convolutional operations with iterative operations. This model is designed to capture spatial and temporal dependencies in time series data, making it well-suited for time series forecasting tasks. In the ConvLSTM model, the input is typically represented as a 3-dimensional tensor that denotes the input sequence. The tensor's dimensions are batch size, time steps, and features. Here, the batch size refers to the number of samples, the time steps represent the number of observations in the series, and the features correspond to the number of variables at each time step [7].

Convolutional LSTM cells extend the traditional LSTM architecture by replacing matrix multiplications with convolution operations within the LSTM cells. The model begins with a series of convolutional layers, followed by a Dropout layer and a set of LSTM layers. Subsequently, there is another Dropout layer and a flattening operation. Following the flattening, a dense set of layers comprising fully connected neurons exists. This modification enables the model to process the input sequence using convolutional filters to capture spatial patterns and time dependencies. These convolutional LSTM cells maintain hidden and cell states, updated at each time step based on the input and the previous hidden state.

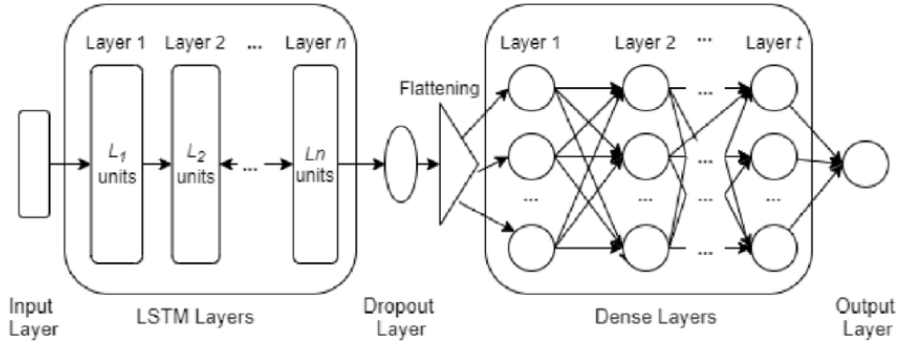


Fig. 10 Enter Caption

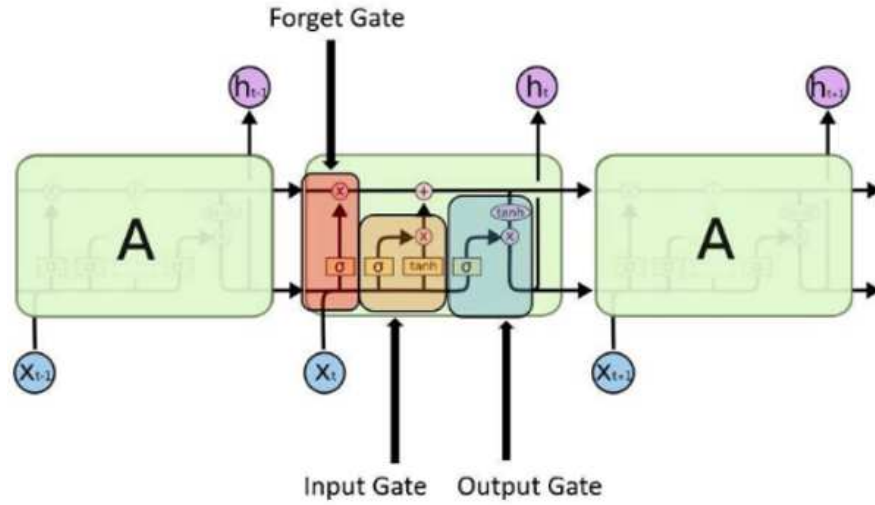


Fig. 11 Peephole LSTM model

3.13 Peephole Long Short – Term Memory

The Peephole LSTM model [10] is a variant of the traditional LSTM model that incorporates additional connections known as "peepholes" into the cell states. These connections enhance the LSTM architecture by allowing the gates access to the cell state, enabling the model to capture long-term dependencies more effectively in time series data. The LSTM cell is the fundamental component of the Peephole LSTM, comprising three main gates: the input gate, forget gate, and output gate. These gates regulate the flow of information within the LSTM cell, determining the amount of information to store or discard from the cell state. In the conventional LSTM model, the gates can only access the current input and the previous hidden state. However, in the Peephole LSTM model, the gates are further augmented by direct access to the cell state. During each time step, the gates of the Peephole LSTM model calculate their respective values based on the current input, the previous hidden state, and the

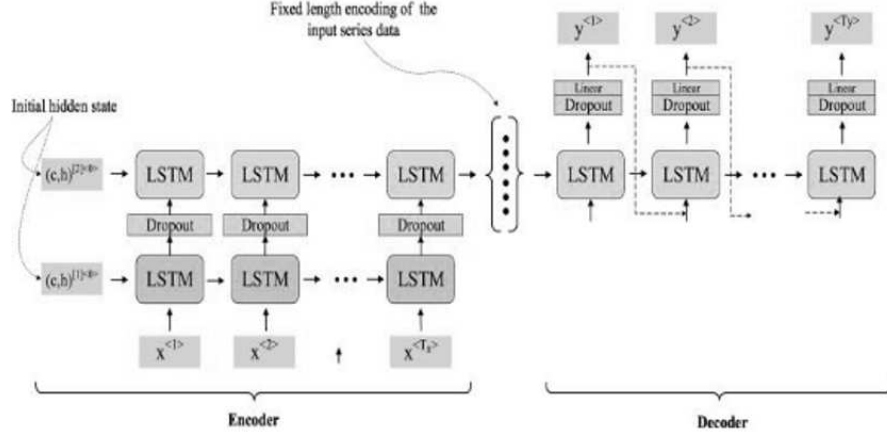


Fig. 12 Encoder-Decoder LSTM model

previous cell state. The input gate determines the extent to which new input should be integrated into the cell state, the forget gate determines which information from the previous cell state should be discarded, and the output gate controls the amount of information output from the current cell state. Once the gate values are computed, the Peephole LSTM model updates the cell state by combining the value of the input gate with the new input, updating the cell state based on the value of the forget gate, and applying the output gate to the current cell state.

3.14 Encoder – Decoder Long Short – Term Memory

Encoder-Decoder LSTM [2] is a model specifically designed to address the sequence-to-sequence (seq2seq) problem, where the length of the input string may differ from the length of the output string. It consists of two models: an encoder model that reads the input string and encodes it into a fixed-length vector and a decoder model that decodes the fixed-length vector to generate the predicted output string.

During the encoding phase, the main objective is to extract additional features and information from the input time series data. The input data, represented as an asymmetric sequence $X = \{x_1, x_2, \dots, x_t\}$, is fed into the encoder, which encodes the sequence into a fixed-length state vector c . This state vector is then used as the input for the decoder. In decoding, the decoder takes the state vector c and generates predictions for the next time series Y by incorporating the input data for the current time step. At each time step, the hidden layer state h is updated as the input data is processed. When the end of the input sequence X is reached, the hidden state h can be viewed as a summary of the entire input sequence.

In the encoder part, the hidden states h_i are computed using the following formula:

$$h_t = f(W^{hh}h_{t-1} + W^{hx}x_t)$$

This simple formula applies appropriate weights to the previous hidden state h_{t-1} and the input vector x_t .

In the decoder part, each hidden state h_{t-1} is calculated as follows:

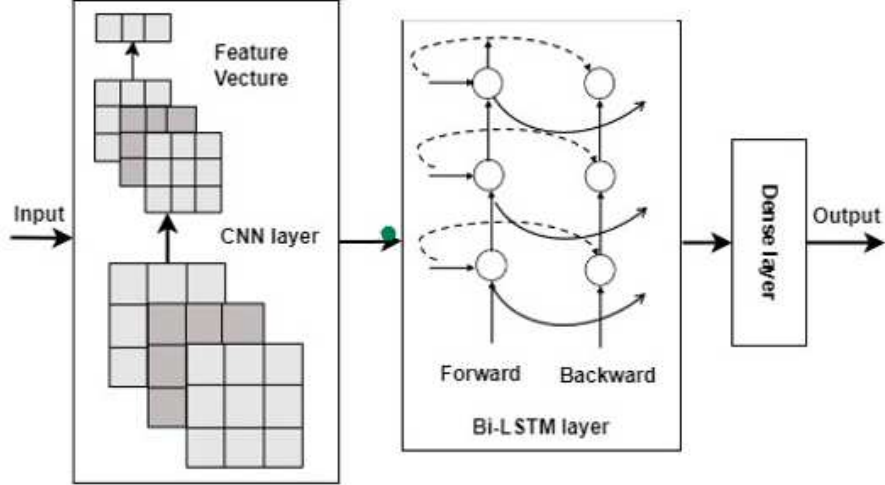


Fig. 13 Architecture of the CNN-BiLSTM network

$$y_t = \text{softmax}(W^s h_t)$$

This formulation applies the softmax activation function to the weighted sum of the hidden state h_t , producing the predicted output y_t .

3.15 Convolutional Bidirectional LSTM Network

The CBiLSTM model [5] comprises convolutional layers that initially take feature embeddings as input. The output of these layers is then aggregated to a smaller size, which is subsequently passed through the BiLSTM layers. The integration of CNN layers provides the model with a powerful capability for local feature extraction. The proposed approach aims to extract internal characteristics of the input time series using the CNN model and capture temporal dependencies using the BiLSTM layer.

The motivation behind combining CNN and BiLSTM lies in the CNN model's exceptional ability to extract features that capture short-term patterns in time series data. Additionally, the BiLSTM network effectively preserves the temporal order between two-dimensional data and mitigates the issue of vanishing gradients. By leveraging the strengths of both CNN and BiLSTM, the CBiLSTM model enables pattern recognition for time series data in both spatial and temporal dimensions [23].

3.16 Optimization Hybrid Long Short – Term Memory–Convolutional Neural Network model

During the training process of a deep learning model, the update of each parameter modifies the data distribution within each layer of the network. As the number of layers increases, the discrepancy in data distribution becomes more pronounced, causing the network to adapt to different distributions with each update continuously. This significantly hampers the training speed of the network. To address this issue, the

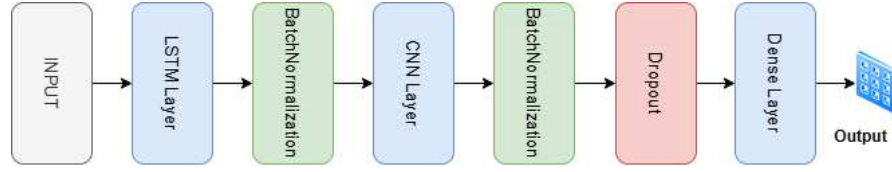


Fig. 14 Improved of Hybrid LSTM – CNN model

proposed Batch Normalization technique enhances model performance by normalizing the features (i.e., the output of each layer after passing through activations) to have a zero mean and unit standard deviation.

Specifically, applying the Batch Normalization technique to the LSTM and CNN layers involves calculating the derivative value and reducing its magnitude according to the derivative direction. However, as the LSTM layer follows the CNN layer, the input data distribution gradually changes due to the weight updates in the previous layers. This leads to a significant disparity between the input distributions of subsequent and preceding layers. Batch Normalization helps rectify this by aligning the data distribution to a normal distribution across all classes, ensuring consistency in the data distribution. Moreover, models with many parameters are susceptible to overfitting during training, resulting in unsatisfactory predictions on the test set. To mitigate this, incorporating the Dropout technique at the end of the model improves generalizability and enhances the overall performance of the model.

4 Evaluation

4.1 Dataset

Our dataset consists of 3 data sets of crypto-currencies, Bitcoin, Ethereum, and Binance, collected daily on the YahooFinance [29]. A short-term dataset from January 1, 2020, and June 1, 2024. For the extended data set, the collection starts from January 1, 2019, until June 1, 2024. Each dataset has ten attributes as follows:

- **Date:** the date to get data with type **Date**.
- **Open, High, Low, Close:** daily opening, high, low, and closing price of the selected cryptocurrency with **Number** type.
- **H - L:** difference of daily high minus daily low of the selected cryptocurrency with type **Number** [11].
- **SMA₇, SMA₁₄, SMA₂₁:** average value of closing prices (Close) in the last 7, 14, and 21 sessions with **Number** type.

The procedure of the model includes the following steps:

- **Step 1:** Process the price data set of cryptocurrencies collected for each cryptocurrency.
- **Step 2:** Preprocess new data from the already collected data set, perform data processing with digital analysis techniques to create new attributes to save in the dataset, including Valuation moving average (SMA) based on closing price, Standard Deviation, and using **dropna** to remove blank and inconsistent data.

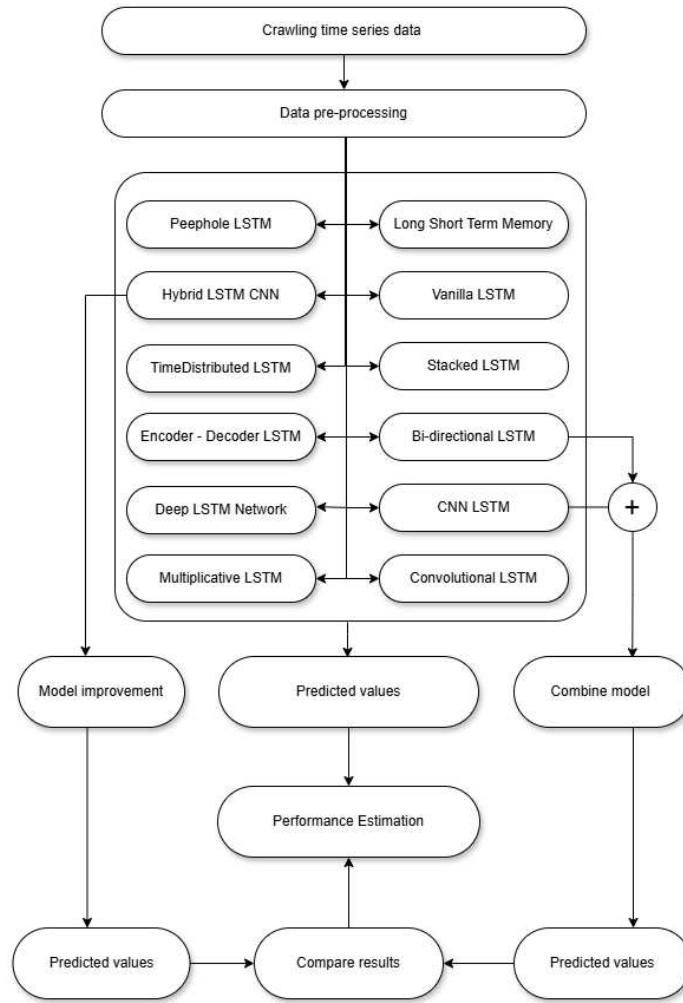


Fig. 15 Experimental procedure

- **Step 3:** Run experiments on each model and provide forecast results.
- **Step 4:** Improve the model and run the combined model to provide all the forecast results.
- **Step 5:** Compare and evaluate the models before and after combining, as well as discuss the findings.
- **Step 6:** Make a forecast and estimate model performance.

4.2 Splitting dataset

In this study, we split the data into 80% training and validation data - 20% test data (8-2). Then we continue to scale 8 - 2 to separate the training dataset and the validation dataset in each dataset of cryptocurrencies [13].

4.3 Splitting dataset

In this study, we used measures such as mean square error (MSE), mean absolute error (MAE), mean square error (RMSE), and absolute percentage error [25]. Mean relative (MAPE) is used to evaluate the error of the prediction result. MAPE, RMSE, and MAE are common indicators to evaluate the accuracy of the proposed model based on the measured value and the estimated value. MSE is a measure of the quality of an estimator - it is always non-negative, and the values are as close to zero as possible. MAE is an estimated index used as a measure. RMSE is used to evaluate the deviation between the observed and actual values [3].

5 Result and Discussion

After running the experiment, we experimented ten times for each model to increase the objectivity of the models and gave the average number of measures. Tables 1, 2 and 3 show the results of the models in 3 different coins: Bitcoin (BTC), Ethereum (ETH) and Binance (BNB). In the short dataset about Bitcoin, the ConvLSTM model demonstrates the most accurate prediction results, as evidenced by the measured values of MAE (974.6644), MAPE (2.0446%), MSE (2114207.4196), and RMSE (1454.0314).

Table 1 Table of results of models on a short dataset about Bitcoin

Model	MAE	MAPE	MSE	RMSE
LSTM	3265.5859	6.3711%	21857379.9333	4675.1877
Multiplicative LSTM	1917.7456	3.8384%	7490448.5267	2736.8684
Vanilla LSTM	1342.2135	2.7768%	4487901.2391	2118.4667
Hybrid LSTM - CNN	1497.4625	3.2388%	4317369.6925	2077.8281
Stacked LSTM	1403.3173	2.8139%	5067115.8149	2251.0255
Timedistributed LSTM	23170.0646	6.1366%	22142742.1879	4705.6075
CNN-LSTM	1777.4084	3.4477%	6571363.3833	2563.4671
Bi-LSTM	1500.0625	3.1147%	5268641.0896	2295.3521
ConvLSTM	974.6644	2.0446%	2114207.4196	1454.0314
Peephole LSTM	1171.3346	2.3870%	3239809.3741	1799.9470
Encoder-Decoder LSTM	1254.9368	2.5812%	3150916.1862	1775.0820
Deep LSTM Network	1534.0059	2.8162%	6516947.7556	2552.8313
CNN-BiLSTM	1667.4096	3.7775%	15213984.7342	2283.4151

Source: This table presents the performance metrics of various models applied to a Bitcoin dataset.

Similarly, in the Ethereum dataset, the Stacked LSTM model outperforms others, yielding the best results with MAE (63.2090), MAPE (2.4372%), MSE (9602.6828), and RMSE (98.0514). In the case of the Binance coin dataset, the Hybrid LSTM-CNN model showcases superior performance with MAE (10.1965), MAPE (2.5846%), MSE (276.4524), and RMSE (16.6269).

Conversely, the LSTM model exhibits the poorest results among the evaluated models across all three datasets. To provide a more comprehensive understanding

Table 2 Table of model results on a short dataset of Ethereum

Model	MAE	MAPE	MSE	RMSE
LSTM	134.0617	5.1788%	37963.4990	194.8422
Multiplicative LSTM	67.7608	2.6513%	10510.7827	102.5221
Vanilla LSTM	66.4614	2.6057%	10810.7732	103.9749
Hybrid LSTM - CNN	79.5166	3.0429%	14768.4060	121.5253
Stacked LSTM	63.2090	2.4372%	9602.6828	98.0514
Timedistributed LSTM	142.4117	5.5426%	38457.6990	196.1063
CNN-LSTM	93.3866	3.6754%	16548.8790	128.6424
Bi-LSTM	67.3773	2.6402%	10635.7972	103.1300
ConvLSTM	96.6375	3.9635%	15647.5022	125.0900
Peephole LSTM	73.2984	2.8330%	12169.7416	110.3166
Encoder-Decoder LSTM	64.1864	2.4857%	10883.8385	104.3256
Deep LSTM Network	70.7806	2.7330%	11119.9757	105.4513
CNN-BiLSTM	64.4816	2.4841%	9608.2787	98.0218

Source: This table presents the performance metrics of various models applied to an Ethereum dataset.

Table 3 Table of results of models on a short dataset about Binance

Model	MAE	MAPE	MSE	RMSE
LSTM	21.4008	5.8568%	943.0867	30.7097
Multiplicative LSTM	15.5113	4.7343%	428.1823	20.6926
Vanilla LSTM	11.1984	2.7503%	364.1439	19.0826
Hybrid LSTM - CNN	10.1965	2.5846%	276.4524	16.6269
Stacked LSTM	12.1042	3.1268%	515.5976	22.7068
Timedistributed LSTM	28.1059	7.4988%	1632.4844	40.4040
CNN-LSTM	14.2435	4.8771%	328.4724	18.1238
Bi-LSTM	10.9359	2.8119%	298.6749	17.2822
ConvLSTM	10.2825	2.9104%	227.5686	17.0854
Peephole LSTM	13.3495	3.1563%	490.2719	22.1421
Encoder-Decoder LSTM	12.3447	3.3491%	386.5844	19.6618
Deep LSTM Network	11.0206	2.7392%	303.2564	17.4143
CNN-BiLSTM	10.4293	2.6873%	292.0193	17.0886

Source: This table presents the performance metrics of various models applied to a Binance dataset.

of the prediction outcomes, we present a comparison of the graphs illustrating the prediction results of the best and worst models for the three datasets below:

Upon analyzing the graphs, it becomes evident that the best-performing models closely predict the actual results. However, the LSTM model occasionally exhibits erroneous trend predictions, deviating from the actual results (represented by the yellow squares). This observation indicates that the LSTM model's characteristic of enabling information sharing proves ineffective for time series forecasting. Consequently, it negatively impacts the model's performance, resulting in the poorest outcomes across all three datasets.

Nonetheless, since the primary objective of prediction is to develop a model with optimal predictability to assist investors in making well-informed, rational, and timely decisions, we conducted experiments by combining proficient predictive models on a

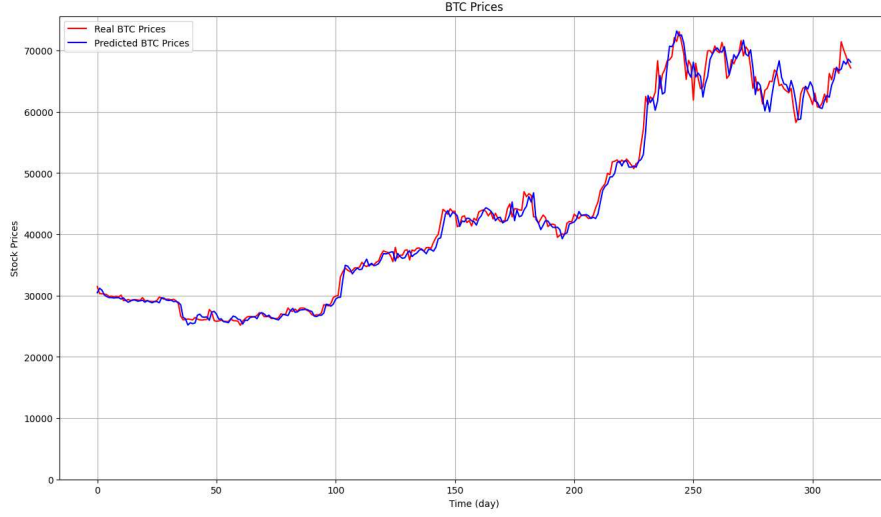


Fig. 16 Graph of prediction results of ConvLSTM model on Bitcoin dataset

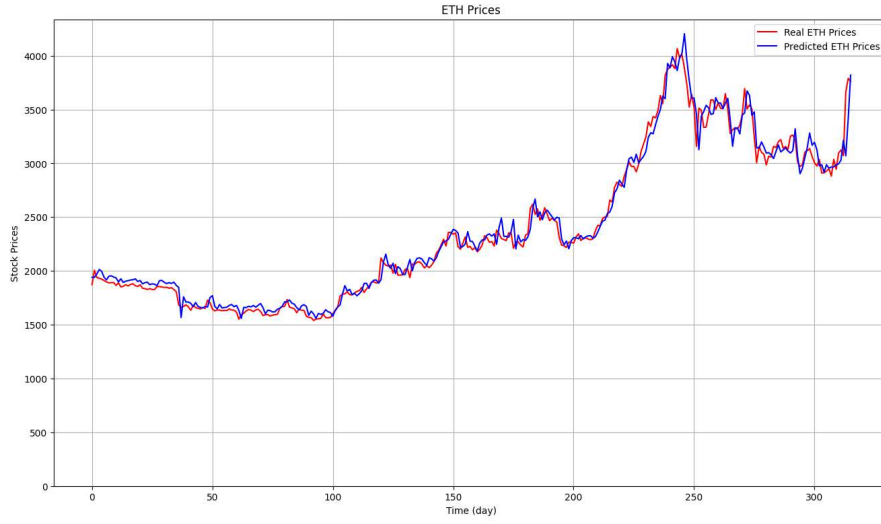


Fig. 17 Graph of prediction results of Stacked LSTM model on Bitcoin dataset

more extended dataset spanning five years. The intention behind this combination was to leverage the strengths of individual prediction models and create a superior composite model that surpasses the predictive capabilities of individual models.

In the results table about the short datasets, our focus centered on three models: BiLSTM, CNN-LSTM, and the combined CBiLSTM model. The evaluation results demonstrate that the CBiLSTM model outperforms the other two models in the short Ethereum and Binance datasets. However, the CBiLSTM model falls short in the Bitcoin dataset compared to the CNN-LSTM model.

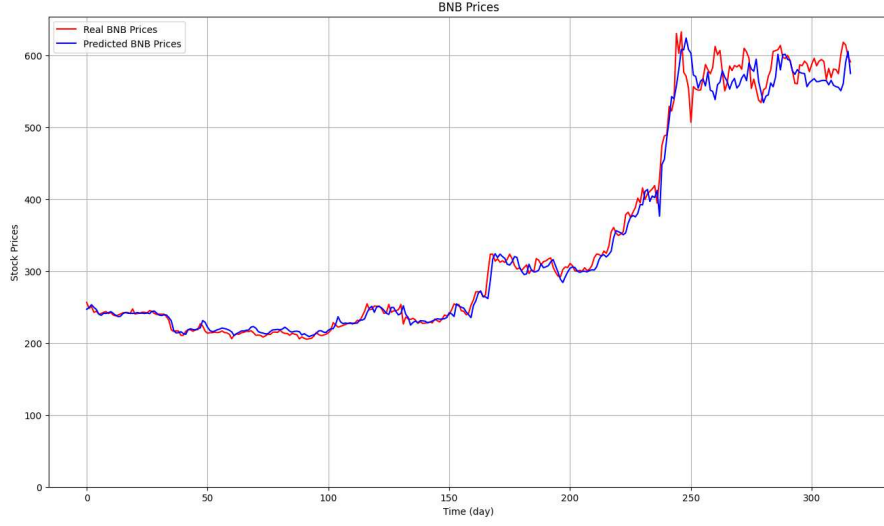


Fig. 18 Graph of prediction results of Hybrid LSTM – CNN model on Binance coin dataset

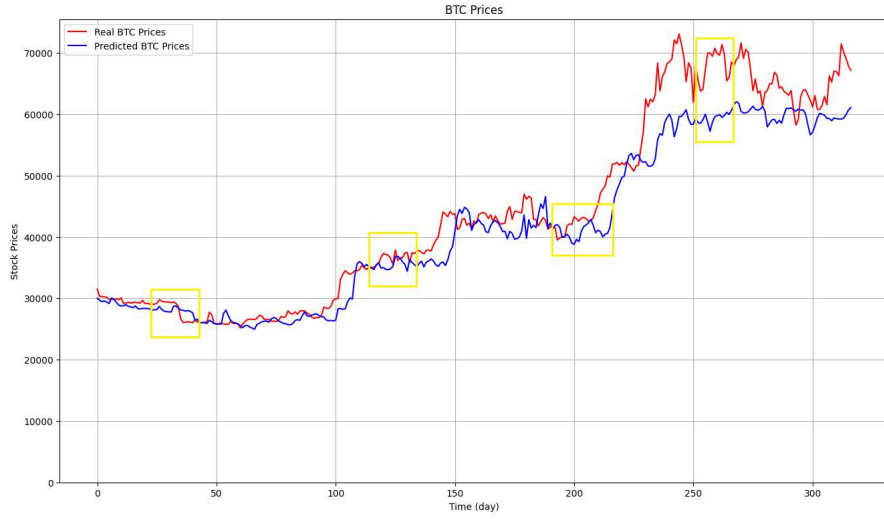


Fig. 19 Graph of LSTM model prediction results on Bitcoin dataset

To account for potential variations and their impact on the predictive models' characteristics, we conducted experiments on a more extended dataset to compare the results of these three models. The subsequent table illustrates the outcomes of these experiments:

Upon analyzing the results table, we observe that combining proficient predictive models leads to improved prediction outcomes. Specifically, for the extended Bitcoin dataset, the combined CNN-LSTM and BiLSTM model exhibits the best results regarding the MAE and MAPE measures, with values of 1202.1470 and 2.8137%,

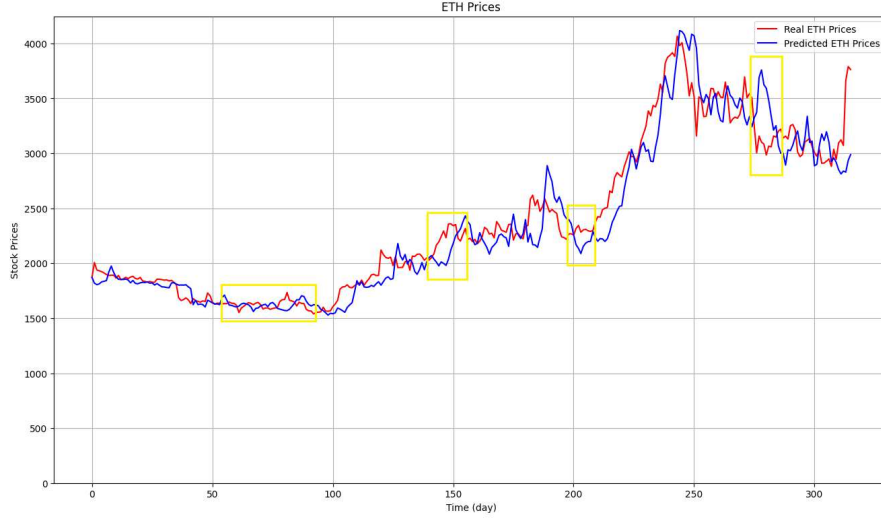


Fig. 20 Graph of prediction results of LSTM model on Ethereum dataset

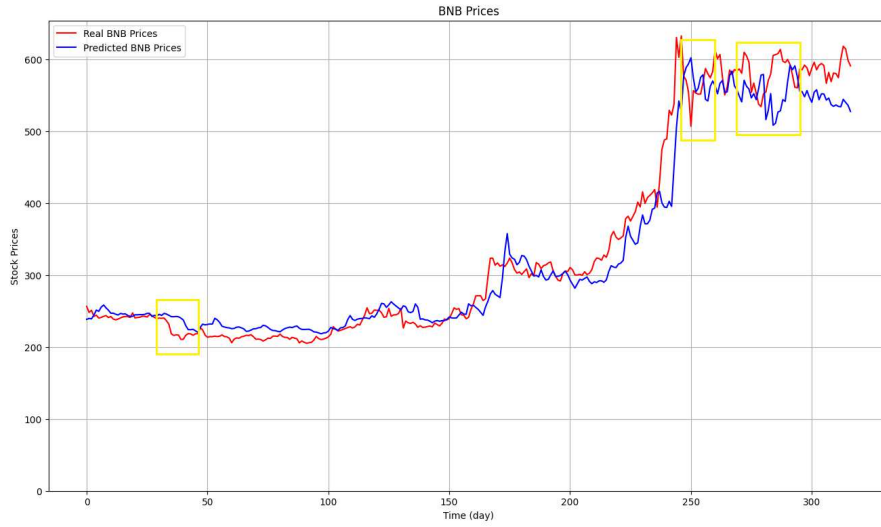


Fig. 21 Graph of prediction results of LSTM model on Ethereum dataset

respectively. However, in terms of the MSE and RMSE measures, the CBiLSTM combined model does not outperform the CNN-LSTM model, which yields the best results with values of MSE – 2988020.1333 and RMSE – 1728.5891. In contrast, for the Ethereum and Binance datasets, the CBiLSTM combined model demonstrates a significant breakthrough, outperforming all other models across all four measures.

By combining the CNN-LSTM model's exceptional feature extraction capabilities, which effectively capture short-term patterns in time series data, with the BiLSTM

Table 4 Results table of 3 models on an extended data set about Bitcoin

Model	MAE	MAPE	MSE	RMSE
BiLSTM	1539.4878	3.6629%	5298097.0218	2301.7595
CNNLSTM	1247.7905	2.8429%	2988020.1333	1728.5891
CBiLSTM	1202.1470	2.8137%	3841765.8003	1960.0423

Source: Performance metrics of various models applied to an extended Bitcoin dataset.

Table 5 Results table of 3 models on an extended data set about Ethereum

Model	MAE	MAPE	MSE	RMSE
BiLSTM	68.0863	2.7501%	10539.2148	102.6607
CNNLSTM	66.2350	2.6522%	14191.7918	118.4304
CBiLSTM	65.6975	2.3498%	12166.8994	101.2660

Source: Performance metrics of various models applied to an extended Ethereum dataset.

Table 6 Results table of 3 models on an extended data set about Binance

Model	MAE	MAPE	MSE	RMSE
BiLSTM	10.4996	2.8501%	290.6078	17.0472
CNNLSTM	10.0171	2.7195%	235.9837	15.3618
CBiLSTM	9.0099	2.4340%	219.4021	14.8122

Source: Performance metrics of various models applied to an extended Binance dataset.

model's efficiency in preserving temporal order and mitigating the issue of vanishing gradients, we have successfully created a highly predictive model.

The graph below shows us that the predictive power of the combined model is very high; there is almost no difference between the predicted result and the actual value (between the blue and red lines). It proves that combining single models that predict well, using the advantages of single models, creates a model that is superior in terms of prediction in time series.

However, if we compare the results in two short and long data sets for the three models, we can see:

- In the Bitcoin dataset: The prediction results of the three models on the extended data set are worse than the predictions on the short data set.
- In the Ethereum dataset: The three models give better results when predicting an extended data set than a short one.
- In the data set about Binance: The two models, BiLSTM and CNN-LSTM, predicted better for the extended data set, but the combined model gave worse results for the short data set.

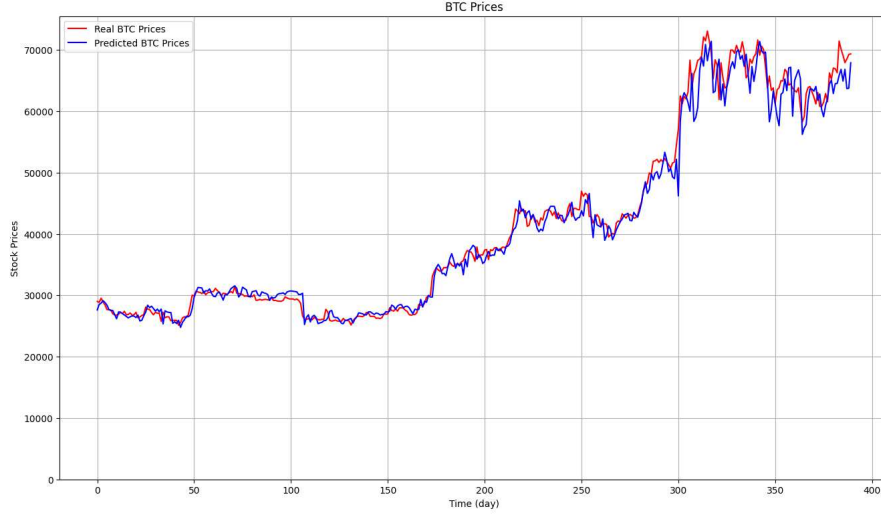


Fig. 22 Graph showing CBiLSTM model prediction results on long data set about Bitcoin

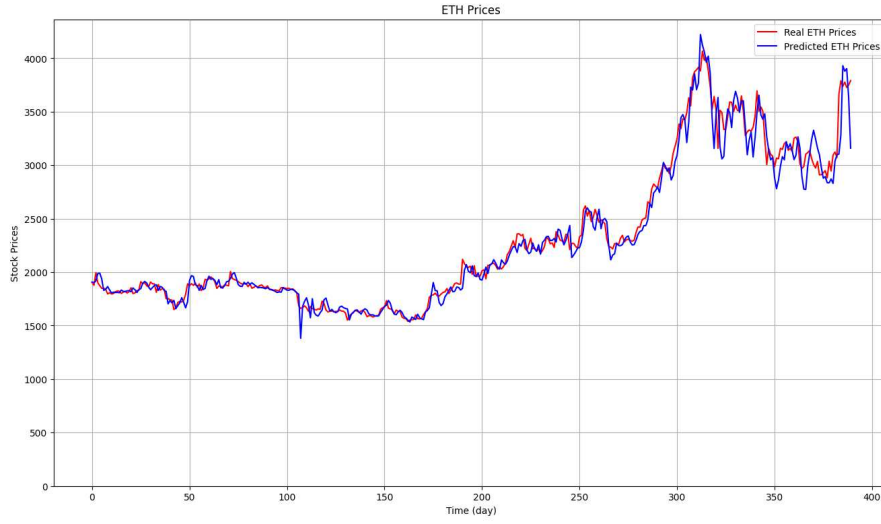


Fig. 23 Graph of CBiLSTM model prediction results on long data set about Ethereum

At the same time, seeing that the Hybrid LSTM - CNN model in the results table on the short set has good results, we propose to improve the above model, and the experimental process gets the following results.

After the improvement of the Hybrid LSTM-CNN model, the results were better, and the metrics were much better than the original model. Visualize the change of the improved model with the graphs below:

Based on the Bitcoin set graph, the improved model has some points (the green zero areas) with a forecast line that is very close to reality, much improved compared

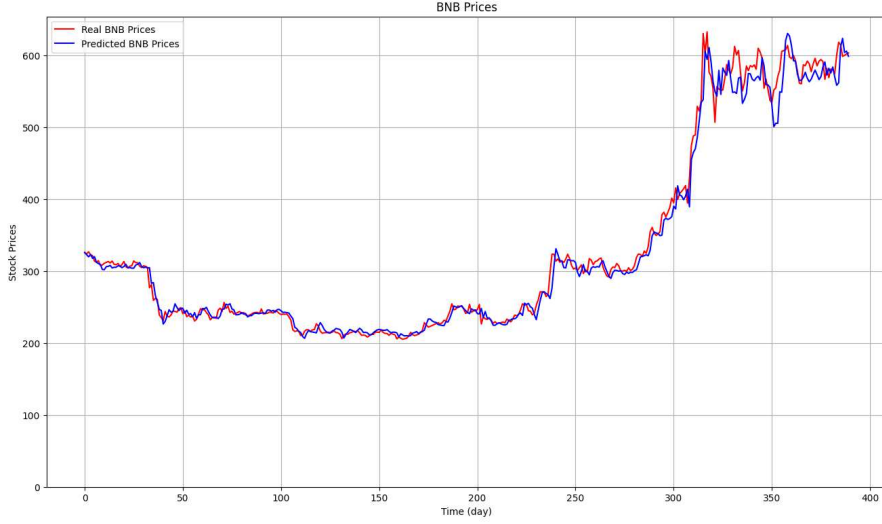


Fig. 24 Graph showing CBiLSTM model prediction results on long dataset about Binance

Table 7 Table of results before and after the improvement of the Hybrid LSTM model - CNN

	BTC	ETH	BNB
MAE Before	1497.4625	79.5166	10.1965
MAE After	1238.1490	71.6459	9.8491
MAPE Before	3.2388%	3.0429%	2.5846%
MAPE After	2.5012%	2.7987%	2.3796%
MSE Before	4317369.6925	14768.4060	276.4524
MSE After	3440989.6168	11998.0923	193.5345
RMSE Before	2077.8281	121.5253	16.6269
RMSE After	1854.9905	109.5358	14.5402

to the graph of the pre-improved model. This improvement has helped the model reduce gradient vanishing and gradient exploding thanks to Batch Normalization while improving the learning speed and reducing overfitting, increasing the stability of the model.

6 Conclusion

This study aims to investigate various LSTM models and their combinations to develop time series prediction models for three datasets: Bitcoin, Ethereum, and Binance coin. Using short and long datasets, we conducted experiments to compare the prediction performance of three models, namely BiLSTM, CNN-LSTM, and the combined CBiLSTM model. Additionally, we made improvements to the Hybrid LSTM-CNN model. The evaluation results demonstrate the enhanced predictive capability of the CBiLSTM combined model for more extended time-series predictions. Furthermore,

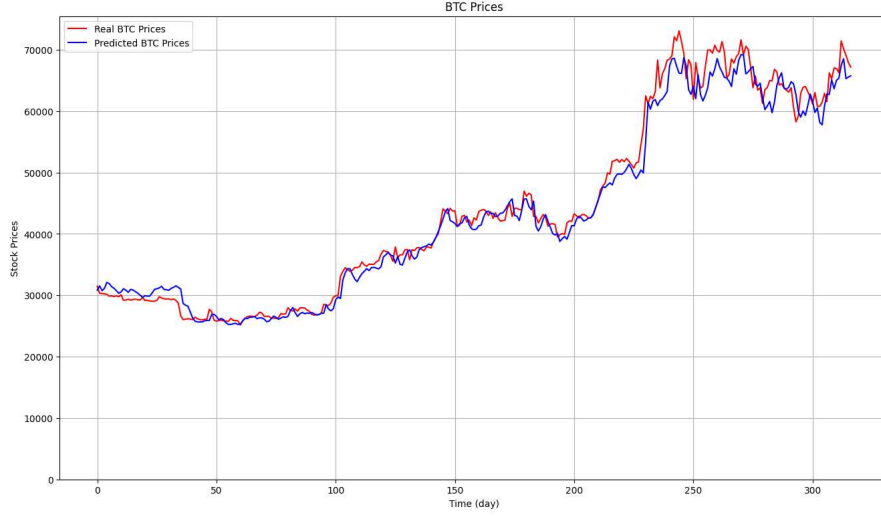


Fig. 25 Forecast graph of Hybrid LSTM-CNN model on Bitcoin dataset

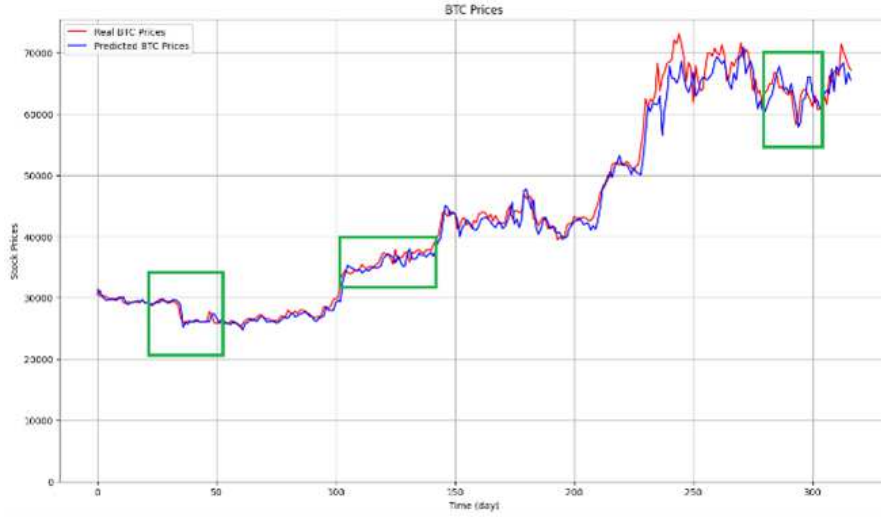


Fig. 26 Forecast graph of Hybrid LSTM-CNN improvement model on Bitcoin dataset

the improved Hybrid LSTM-CNN model exhibits superior performance compared to its previous version. Consequently, the combination of models effectively addresses the limitations of individual models. It leverages the strengths of each, resulting in enhanced predictions that assist investors in making more informed, rational, and accurate decisions. In the future, we will concentrate on advancing this research by developing novel forecasting methodologies rooted in Kolmogorov-Arnold Networks (KAN) architectures, poised as promising alternatives to Multi-Layer Perceptrons

(MLPs). Our goal is to enhance predictive accuracy across diverse cryptocurrency markets, equipping investors with robust decision-making tools.

Acknowledgments

This research was supported by The VNUHCM-University of Information Technology's Scientific Research Support Fund.

References

- [1] Fachrizal Aksan et al. "CNN-LSTM vs. LSTM-CNN to Predict Power Flow Direction: A Case Study of the High-Voltage Subnet of Northeast Germany". In: *Sensors* 23.2 (2023). ISSN: 1424-8220. DOI: [10.3390/s23020901](https://doi.org/10.3390/s23020901). URL: <https://www.mdpi.com/1424-8220/23/2/901>.
- [2] Khder Alakkari et al. "Forecasting COVID-19 Infection Using Encoder-Decoder LSTM and Attention LSTM Algorithms". In: (Jan. 2023), pp. 20–33. DOI: [10.54216/JISIoT.080202](https://doi.org/10.54216/JISIoT.080202).
- [3] P. Bhandari. *Standard Deviation*. [Online; Accessed: 2024-10-30]. June 2023. URL: <https://www.scribbr.com/statistics/standard-deviation/#:~:text=The%20standard%20deviation%20is%20the,clustered%20close%20to%20the%20mean.>
- [4] A. Biswal. *Recurrent Neural Network (RNN) Tutorial: Types, Examples, LSTM and More*. [Accessed: ngày tháng năm]. 2023. URL: <https://www.guru99.com/recurrent-neural-network-tutorial.html>.
- [5] Bharat Bohara et al. *Short-Term Aggregated Residential Load Forecasting using BiLSTM and CNN-BiLSTM*. Feb. 2023. DOI: [10.48550/arXiv.2302.05033](https://arxiv.org/abs/2302.05033).
- [6] J. Brownlee. *How to Use the TimeDistributed Layer in Keras*. [Online; Accessed: 2024-10-30]. 2017. URL: <https://machinelearningmastery.com/use-timedistributed-layer-keras/>.
- [7] Rogério Luís de C. Costa. "Convolutional-LSTM networks and generalization in forecasting of household photovoltaic generation". In: *Engineering Applications of Artificial Intelligence* 116 (2022), p. 105458. ISSN: 0952-1976. DOI: <https://doi.org/10.1016/j.engappai.2022.105458>. URL: <https://www.sciencedirect.com/science/article/pii/S0952197622004481>.
- [8] Changchun Cai et al. "Short-Term Load Forecasting Based on Deep Learning Bidirectional LSTM Neural Network". In: *Applied Sciences* 11.17 (2021). ISSN: 2076-3417. DOI: [10.3390/app11178129](https://doi.org/10.3390/app11178129). URL: <https://www.mdpi.com/2076-3417/11/17/8129>.
- [9] Ziliang Chen and Zhanfu Yang. "Graph Neural Reasoning May Fail in Certifying Boolean Unsatisfiability". In: *ArXiv abs/1909.11588* (2019). URL: <https://api.semanticscholar.org/CorpusID:203591909>.
- [10] R. Dathan and Subramanian Shanmuga priya. "Yield Forecast of Soyabean Crop Using Peephole LSTM Framework". In: Nov. 2022, pp. 261–270. ISBN: 978-981-19-3147-5. DOI: [10.1007/978-981-19-3148-2_22](https://doi.org/10.1007/978-981-19-3148-2_22).
- [11] DNS Education. *ng SMA là gì?* [Accessed: 2024-10-30]. n.d. URL: <https://www.dnse.com.vn/hoc/duong-sma-la-gi>.

- [12] Lei Fu. “Time Series-oriented Load Prediction Using Deep Peephole LSTM”. In: *2020 12th International Conference on Advanced Computational Intelligence (ICACI)*. 2020, pp. 86–91. DOI: [10.1109/ICACI49185.2020.9177688](https://doi.org/10.1109/ICACI49185.2020.9177688).
- [13] C. M. Hiu. *Scaling vs normalization*. Truy cập ngày: 30 tháng 10 2024. Sept. 2021. URL: <https://viblo.asia/p/scaling-vs-normalization-oOVYJJz58W>.
- [14] Greg Van Houdt, Carlos Mosquera, and Gonzalo Nápoles. “A review on the long short-term memory model”. In: *Artificial Intelligence Review* 53 (2020), pp. 5929–5955. URL: <https://api.semanticscholar.org/CorpusID:218605692>.
- [15] Jinah Kim and Nammee Moon. “BiLSTM model based on multivariate time series data in multiple field for forecasting trading area”. In: *Journal of Ambient Intelligence and Humanized Computing* (2019). URL: <https://api.semanticscholar.org/CorpusID:199003740>.
- [16] P. Kumari and D. Toshniwal. “Long short-term memory–convolutional neural network based deep hybrid approach for solar irradiance”. In: *Journal of Cleaner Production* 286 (2021), p. 125438. DOI: [10.1016/j.jclepro.2020.125438](https://doi.org/10.1016/j.jclepro.2020.125438).
- [17] Z. Liao et al. “SIRVD-DL: A COVID-19 deep learning prediction model based on time-dependent SIRVD”. In: *Computers in Biology and Medicine* 138 (2021), p. 104868.
- [18] Narendran Sobanapuram Muruganandam and Umamakeswari Arumugam. “Seminal Stacked Long Short-Term Memory (SS-LSTM) Model for Forecasting Particulate Matter (PM_{2.5} and PM₁₀)”. In: *Atmosphere* 13.10 (2022). ISSN: 2073-4433. DOI: [10.3390/atmos13101726](https://doi.org/10.3390/atmos13101726). URL: <https://www.mdpi.com/2073-4433/13/10/1726>.
- [19] Mohil Maheshkumar Patel et al. “A Deep Learning-based Cryptocurrency Price Prediction Scheme for Financial Institutions”. In: *Journal of Information Security and Applications* 55 (2020), p. 102583. ISSN: 2214-2126. DOI: <https://doi.org/10.1016/j.jisa.2020.102583>. URL: <https://www.sciencedirect.com/science/article/pii/S2214212620307535>.
- [20] Md Fazle Rabby et al. “Stacked LSTM based deep recurrent neural network with kalman smoothing for blood glucose prediction”. In: *BMC Medical Informatics and Decision Making* 21 (2021). URL: <https://api.semanticscholar.org/CorpusID:231632839>.
- [21] *RNN with Keras: Understanding Computations*. [Accessed: 2024-10-30]. n.d. URL: [%7BURL%20of%20the%20document%20here%7D](#).
- [22] Phumudzo Lloyd Seabe, Claude Rodrigue Bambe Moutsinga, and Edson Pindza. “Forecasting Cryptocurrency Prices Using LSTM, GRU, and Bi-Directional LSTM: A Deep Learning Approach”. In: *Fractal and Fractional* 7.2 (2023). ISSN: 2504-3110. DOI: [10.3390/fractalfract7020203](https://doi.org/10.3390/fractalfract7020203). URL: <https://www.mdpi.com/2504-3110/7/2/203>.
- [23] Liqun Shan et al. “CNN-BiLSTM Hybrid Neural Networks with Attention Mechanism for Well Log Prediction”. In: *Journal of Petroleum Science and Engineering* 205 (Apr. 2021), p. 108838. DOI: [10.1016/j.petrol.2021.108838](https://doi.org/10.1016/j.petrol.2021.108838).
- [24] Liqun Shan et al. “CNN-BiLSTM hybrid neural networks with attention mechanism for well log prediction”. In: *Journal of Petroleum Science and Engineering* 205 (2021), p. 108838. ISSN: 0920-4105. DOI: <https://doi.org/10.1016/j.petrol.2021.108838>.

- petrol.2021.108838. URL: <https://www.sciencedirect.com/science/article/pii/S092041052100499X>.
- [25] Sima Siami-Namini, Neda Tavakoli, and Akbar Siami Namin. “A Comparison of ARIMA and LSTM in Forecasting Time Series”. In: *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 2018, pp. 1394–1401. DOI: [10.1109/ICMLA.2018.00227](https://doi.org/10.1109/ICMLA.2018.00227).
 - [26] Nguyn Tn Tun. *Bài 6: Convolutional Neural Network*. [Accessed: 2024-10-30]. 2023. URL: [%7Bhttps://nttuan8.com/bai-6-convolutional-neural-network/%7D](https://nttuan8.com/bai-6-convolutional-neural-network/).
 - [27] Jianxin Wu. “Introduction to Convolutional Neural Networks”. In: 2017. URL: <https://api.semanticscholar.org/CorpusID:36074296>.
 - [28] Anita Yadav, C K Jha, and Aditi Sharan. “Optimizing LSTM for time series prediction in Indian stock market”. In: *Procedia Computer Science* 167 (2020). International Conference on Computational Intelligence and Data Science, pp. 2091–2100. ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2020.03.257>. URL: <https://www.sciencedirect.com/science/article/pii/S1877050920307237>.
 - [29] *Yahoo! Finance*. [Online; Accessed: 2024-10-30]. 2023. URL: https://finance.yahoo.com/?guccounter=1&guce_referrer=aHR0cHM6Ly93d3cuZ29vZ2xlLmNvbS8&guce_referrer_sig=AQAAAJRWwAK0boZ_da2RpwdhT4z_T-xizYe1qN7ELgjlj_HDYfgjAeHCEbFZbLCOMqRXklzfVR349HZbD9o71SbtSf-xBy120isbFkTIY5LtEZcqsHKt7ksG0toHY8cJ3Ej6vkEX1O7mO1NCHPZb5D.